

title: 模块二：方法与心法 —— 结构化运营分析框架 type: 方法论 tags: [DRG, DIP, 病种分析, 机器学习, 课件, Dashboard, SQL] date: 2026-04-13

模块二：方法与心法 —— 结构化运营分析框架

本模块将深入探讨在医院运营管理及DRG/DIP支付制度改革背景下，如何构建结构化、体系化且可落地的分析框架。本模块内容极具深度与广度，涵盖了从理论模型构建、指标体系拆解、SQL底层数据拉取到高阶统计建模与机器学习预测的全链路闭环。我们将以学术级的严谨态度，辅以详实的数据示例和可执行的代码片段，为您展现医疗数据分析的巅峰全景。

2.1 DRG 四维分析框架 (DRG 4D Analytical Framework)

在DRG（疾病诊断相关分组）支付体系下，医院或科室的综合运营能力不再能够通过单一的“收入”指标来衡量。我们需要建立一个多维度的评价体系，以全面反映医疗服务的全貌。DRG四维分析框架（能力 Capability、效率 Efficiency、质量 Quality、成本/经济 Cost/Economy）正是为此而生，它源于绩效考核的要求，并升华为精细化管理的圭臬。

2.1.1 框架理论解析

- 1. 能力 (Capability)** * **核心逻辑**：反映医院或科室收治患者的疑难危重程度及疾病覆盖面。 * **关键指标**： * **DRG组数**：科室收治病例所覆盖的DRG组数量。组数越多，说明诊疗范围越广，综合能力越强。 * **CMI值 (Case-Mix Index, 病例组合指数)**：衡量收治病例总体平均难度的指标。公式：
$$\text{CMI} = \frac{\sum (\text{各DRG组病例数} \times \text{该DRG组相对权重})}{\text{总病例数}}$$
。CMI>1通常意味着难度高于平均水平。 * **总权重 (Total Weight)**：总权重 = 出院人次 × CMI。反映总产出。
- 2. 效率 (Efficiency)** * **核心逻辑**：反映医疗资源的周转速度及利用效率。 * **关键指标**： * **时间消耗指数 (Time Consumption Index)**：
$$\text{TCI} = \frac{\sum (\text{各DRG组病例实际住院天数} \times \text{该DRG组标准平均住院日权重})}{\text{总病例数} \times \text{整体平均住院日}}$$
。简化理解：科室实际住院日与同类型病例全市/全省平均水平的比值。TCI < 1 表示效率高。 * **床位使用率、床位周转次数**：传统的效率指标，结合DRG可进行标化比较。
- 3. 质量 (Quality)** * **核心逻辑**：医疗安全是底线，反映治疗过程的安全性和有效性。 * **关键指标**： * **中低风险组死亡率**：本不该发生死亡的病例发生了死亡，是极严重的质量预警指标。 * **非**

计划重返率 (如31天内非计划重返住院率、再手术率): 反映治疗不彻底或并发症。* **院内感染率、并发症发生率。**

4. 成本/经济 (Cost / Economy) * 核心逻辑: 在医保基金定额支付的约束下, 科室/医院的成本控制能力及盈利空间。* **关键指标:** * **费用消耗指数 (Cost Consumption Index):** 类似时间消耗指数, 计算实际费用与标杆费用的比值。CCI < 1 表示费用控制好。* **DRG结余/超支率:**
$$\frac{\text{DRG医保拨付额} - \text{实际医疗总费用}}{\text{实际医疗总费用}}$$
。正值为结余, 负值为超支。注意: 这里的实际医疗总费用应尽可能接近真实成本, 而非医院内部定价的收费。

2.1.2 部门级 DRG 全景 Dashboard 设计 (Python Dash 示例)

为了直观监控和管理这四大维度, 我们可以使用 Python Dash 构建一个交互式的数据看板。以下代码提供了一个企业级Dashboard的核心结构设计, 通过生成假数据展示科室在四维框架下的表现。

```
import dash
from dash import dcc, html
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import numpy as np
from dash.dependencies import Input, Output

# --- 1. 生成模拟的科室级DRG运营数据 ---
np.random.seed(42)
departments = ['心血管内科', '呼吸内科', '神经外科', '骨科', '普通外科', '肿瘤科', '儿科', '妇产科']
n_deps = len(departments)

df_dept = pd.DataFrame({
    'Department': departments,
    # 能力指标
    'Discharges': np.random.randint(1000, 5000, n_deps),
    'DRG_Groups': np.random.randint(50, 200, n_deps),
    'CMI': np.random.uniform(0.8, 2.5, n_deps).round(2),
    # 效率指标
    'Time_Index': np.random.uniform(0.7, 1.3, n_deps).round(2),
    'ALOS': np.random.uniform(4.0, 12.0, n_deps).round(1), # Average Length of Stay
    # 质量指标
    'LowRisk_Mortality': np.random.uniform(0, 0.5, n_deps).round(2), # %
    'Reapmission_31d': np.random.uniform(1.0, 5.0, n_deps).round(2), # %
    # 经济/成本指标
    'Cost_Index': np.random.uniform(0.8, 1.4, n_deps).round(2),
    'Avg_Cost': np.random.uniform(8000, 35000, n_deps).round(0),
```

```

    'Profit_Margin': np.random.uniform(-10, 20, n_deps).round(1) # % 盈余率
})

# 计算综合得分（简化版：指标归一化后加权）
df_dept['Score'] = (
    (df_dept['CMI'] / df_dept['CMI'].max()) * 30 +
    (1 / df_dept['Time_Index']) * 20 +
    (1 / df_dept['Cost_Index']) * 30 +
    (100 - df_dept['LowRisk_Mortality']*100) * 0.2
).round(1)

# --- 2. 初始化 Dash 应用 ---
app = dash.Dash(__name__)
app.title = "DRG 四维科室运营全景看板"

# --- 3. 布局设计 ---
app.layout = html.Div(style={'fontFamily': 'Arial, sans-serif', 'padding': '20px'}, children=[
    html.H1("🏥 医院科室 DRG 四维运营分析看板 (Capability, Efficiency, Quality, Cost)",
            style={'textAlign': 'center', 'color': '#2c3e50'}),

    # 下拉菜单选择科室
    html.Div([
        html.Label("选择科室分析:"),
        dcc.Dropdown(
            id='dept-dropdown',
            options=[{'label': i, 'value': i} for i in departments],
            value='心血管内科',
            style={'width': '50%'}
        )
    ], style={'paddingBottom': '20px'}),

    # 核心指标卡片区
    html.Div(id='kpi-cards', style={'display': 'flex', 'justifyContent': 'space-between', 'margin: 20px 0'}),

    # 图表区：四象限图与雷达图并排
    html.Div([
        # 散点图（波士顿矩阵变体：时间消耗 vs 费用消耗）
        html.Div(dcc.Graph(id='scatter-matrix'), style={'width': '55%', 'display': 'inline-block', 'margin-right: 10px'}),

        # 雷达图（科室四维能力多边形）
        html.Div(dcc.Graph(id='radar-chart'), style={'width': '40%', 'display': 'inline-block'})
    ])
])

# --- 4. 回调函数实现动态交互 ---
@app.callback(

```

```

[Output('kpi-cards', 'children'),
 Output('scatter-matrix', 'figure'),
 Output('radar-chart', 'figure')],
[Input('dept-dropdown', 'value')]
)
def update_dashboard(selected_dept):
    # 1. 提取选中科室的数据
    d = df_dept[df_dept['Department'] == selected_dept].iloc[0]

    # 定义卡片样式
    card_style = {'padding': '20px', 'borderRadius': '10px', 'boxShadow': '0 4px 8px 0 rgba(0,0,0,0.1)'}

    # 2. 生成 KPI 卡片
    kpi_html = [
        html.Div([html.H3("💪 能力 (CMI)"), html.H2(f"{d['CMI']}", style={'color': '#2980b9'})], card_style),
        html.Div([html.H3("⚙️ 效率 (TCI)"), html.H2(f"{d['Time_Index']}", style={'color': '#27ae60'})], card_style),
        html.Div([html.H3("🛡️ 质量 (低风险死亡)"), html.H2(f"{d['LowRisk_Mortality']}"%, style={'color': '#27ae60'})], card_style),
        html.Div([html.H3("💰 经济 (盈余率)"), html.H2(f"{d['Profit_Margin']}"%, style={'color': '#27ae60'})], card_style)
    ]

    # 3. 生成 效率-成本 散点矩阵图 (所有科室全局视图, 高亮选中科室)
    fig_scatter = px.scatter(
        df_dept, x='Cost_Index', y='Time_Index',
        size='Discharges', color='Department',
        hover_name='Department',
        title="双维消耗指数矩阵 (全局科室对比)",
        labels={'Cost_Index': '费用消耗指数 (CCI)', 'Time_Index': '时间消耗指数 (TCI)'}
    )
    # 添加基准线
    fig_scatter.add_hline(y=1, line_dash="dash", line_color="red")
    fig_scatter.add_vline(x=1, line_dash="dash", line_color="red")
    # 高亮选中的科室
    fig_scatter.add_annotation(
        x=d['Cost_Index'], y=d['Time_Index'],
        text=f"👉 目标: {selected_dept}",
        showarrow=True, arrowhead=1, ax=50, ay=-50, bordercolor="#c7c7c7", borderwidth=2, borderdash=[5, 5])
    )

    # 4. 生成选定科室的 四维雷达图 (归一化处理以便同图展示)
    max_cmi, max_tci, max_cci, max_pm = df_dept['CMI'].max(), df_dept['Time_Index'].max(), df_dept['Cost_Index'].max(), df_dept['Profit_Margin'].max()

    # 注意: TCI和CCI越小越好, 因此使用 1/值 进行标准化展示
    categories = ['能力(CMI标化)', '效率(1/TCI标化)', '经济(1/CCI标化)', '盈余率标化', '综合规模标化']
    values = [
        d['CMI'] / max_cmi,
        (1/d['Time_Index']) / (1/df_dept['Time_Index'].min()),
        (1/d['Cost_Index']) / (1/df_dept['Cost_Index'].min()),
        d['Profit_Margin'] / max_pm
    ]

```

```

d['Profit_Margin'] / max_pm if max_pm > 0 else 0,
d['Discharges'] / df_dept['Discharges'].max()
]

fig_radar = go.Figure()
fig_radar.add_trace(go.Scatterpolar(
    r=values + [values[0]], # 闭合多边形
    theta=categories + [categories[0]],
    fill='toself',
    name=selected_dept,
    line_color='indigo'
))
fig_radar.update_layout(
    polar=dict(radialaxis=dict(visible=True, range=[0, 1])),
    showlegend=False,
    title=f"{selected_dept} 综合运营雷达图"
)

return kpi_html, fig_scatter, fig_radar

if __name__ == '__main__':
    app.run_server(debug=True, port=8050)

```

看板运行逻辑解析：该Python脚本通过构建内存中的虚拟数据集，利用Dash组件实现了实时联动的分析界面。散点矩阵图（CCI vs TCI）极其关键：*** 左下象限（低消耗，低成本）：**优势学科，应重点倾斜资源。*** 右上象限（高消耗，高成本）：**劣势学科，临床路径急需整改，属于DRG下的“流血”科室。

2.2 DIP 三层穿透模型 (DIP Three-Tier Penetration Model)

按病种分值付费（DIP）是中国本土首创的医保支付方式，其核心在于“基于历史数据的真实世界病例组合”。与DRG人为预先设定规则不同，DIP是自下而上通过“主诊断+主手术操作”聚合生成的。这使得DIP的病种数量极其庞大（通常上万组），因此对其分析必须采用“由宏入微、逐层穿透”的**三层穿透模型：宏观（Macro）、中观（Meso）、微观（Micro）**。

2.2.1 三层穿透模型拆解

第一层：宏观层 (Macro - 院级/整体概览)

目标：掌握全局医保结算状态，判断整体盈亏风险，评估医院层面的病种结构健康度。**分析维度：***** 全院拨付率与超支结余：**实际总拨付 / 实际总花费。*** CMI与总分值规模：**全院CMI值（分

值/病例数标化) 年度趋势。* **基础病种与核心病种结构**: 符合DIP核心目录的病例占比。* **费用畸变率 (合规率/倍率整体情况)**: 高倍率 (大于标准费用一定倍数, 如2倍、3倍) 和低倍率 (极低费用) 病例的总体占比。

第二层: 中观层 (Meso - 科室/MDC/DIP病种群组)

目标: 定位“出血点”与“利润牛”, 找到导致超支或结余的主要责任科室及关键病种群。 **分析维度**: * **科室盈亏贡献度 (Pareto 帕累托分析)**: 哪些科室贡献了80%的结余? 哪些科室造成了80%的亏损? * **高优病种 vs 劣势病种盘点 (波士顿矩阵)**: 按“分值 (价格)”与“病例量 (市场份额)”对科室下的病种进行四象限划分。* **资源消耗结构**: 按科室拆解药占比、耗占比、医疗服务费占比。

第三层: 微观层 (Micro - 细分病种/单体病例追踪)

目标: 针对具体的亏损或高倍率病种, 下探至明细清单与患者个体, 找出超支的具体发生源 (是哪个医生? 用了什么高值耗材? 是否发生并发症?)。 **分析维度**: * **倍率分析 (Compliance Ratio Analysis)**: 这是DIP极具特色的分析。* **常规病例 (如0.5 ~ 2倍)**: 按分值正常结算, 考验临床路径执行力。* **低倍率病例 (如 < 0.5倍)**: 住院天数极短或花费极低, 通常只结算实际费用, 甚至被剔除。需分析是否属于“未达入院标准而收治 (挂床/冲量)”。* **高倍率病例 (如 > 2倍/3倍/特例单议)**: 花费远超基准。可能因为合并重症、使用极高值耗材。此类病例若不能成功“特例单议”, 将给科室带来巨大亏损。* **病历质控与编码核查**: 是否有高编 (Upcoding)、低编 (Downcoding) 或诊断遗漏导致未能入组更高分值的DIP。

2.2.2 底层 SQL 查询实战: 从流水账到三层模型数据

为支持上述三层穿透, 数据工程师需要从底层的医院HIS计费系统和病案首页库中提取数据。以下是一组结构化的SQL查询示例 (基于标准化的关系型数据库结构, 如 PostgreSQL/SQL Server), 展示如何计算倍率并分层。

前提设定假想表结构: * **med_records (病案首页表)**: case_id, dept_id, doctor_id, discharge_date, main_diag_code, main_oper_code, dip_code * **billing_details (费用明细表)**: case_id, item_type (药品/耗材/服务), amount * **dip_catalog (DIP分值目录表)**: dip_code, dip_name, base_score (基础分值), benchmark_cost (基准费用/次均费用定额)

```
-- =====
-- 步骤 1: 准备基础宽表 (将首页与总费用、DIP标准进行连接)
-- 这一步生成病例级别的微观明细 (Micro Level Data)
-- =====

WITH CaseLevelData AS (
    SELECT
```

```

    m.case_id,
    m.dept_id,
    m.doctor_id,
    m.dip_code,
    c.dip_name,
    c.base_score,
    c.benchmark_cost,
    -- 计算单病例总费用
    COALESCE(SUM(b.amount), 0) AS total_actual_cost,
    -- 计算倍率 (实际费用 / 基准定额费用)
    CASE
        WHEN c.benchmark_cost > 0 THEN COALESCE(SUM(b.amount), 0) / c.benchmark_cost
        ELSE 0
    END AS cost_ratio
FROM med_records m
LEFT JOIN dip_catalog c ON m.dip_code = c.dip_code
LEFT JOIN billing_details b ON m.case_id = b.case_id
WHERE m.discharge_date BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY m.case_id, m.dept_id, m.doctor_id, m.dip_code, c.dip_name, c.base_score, c.benchma
),

-- =====
-- 步骤 2: 计算倍率分组并预估盈亏 (基于本地DIP政策假定)
-- 假定:
-- 1. ratio < 0.5 为极低倍率, 按实际费用支付 (盈亏=0)
-- 2. 0.5 <= ratio <= 2 为正常倍率, 按标准定额支付 (盈亏 = 定额 - 实际)
-- 3. ratio > 2 为高倍率特例, 超出部分打折支付或需审批, 为简化, 此处假定直接亏损(定额-实际)
-- =====
CaseProfitLoss AS (
    SELECT
        *,
        CASE
            WHEN cost_ratio < 0.5 THEN '低倍率 (<0.5)'
            WHEN cost_ratio BETWEEN 0.5 AND 2.0 THEN '常规病例 (0.5-2.0)'
            WHEN cost_ratio BETWEEN 2.0001 AND 3.0 THEN '高倍率 (2.0-3.0)'
            ELSE '极高倍率 (>3.0)'
        END AS ratio_tier,

        CASE
            WHEN cost_ratio < 0.5 THEN 0 -- 实际报销实际, 不亏不赚
            ELSE benchmark_cost - total_actual_cost -- 定额结算, 节余为正, 超支为负
        END AS estimated_profit
    FROM CaseLevelData
)

-- =====
-- 步骤 3: 提取宏观层 (Macro Level) 统计

```

```

-- 目标：全院层面的DIP总体运营情况
-- =====
-- SELECT
--     COUNT(case_id) AS total_cases,
--     SUM(base_score) AS total_scores,
--     SUM(total_actual_cost) AS total_hospital_cost,
--     SUM(estimated_profit) AS net_hospital_profit,
--     SUM(CASE WHEN estimated_profit > 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(case_id) AS profit
-- FROM CaseProfitLoss;

-- =====
-- 步骤 4：提取中观层（Meso Level）统计 —— 科室维度的倍率分布与盈亏
-- 目标：找出哪些科室亏损严重，其倍率结构是怎样的？
-- =====
SELECT
    dept_id,
    COUNT(case_id) AS case_volume,
    SUM(base_score) AS dept_total_score,
    SUM(estimated_profit) AS dept_net_profit,
    -- 倍率结构分布
    SUM(CASE WHEN ratio_tier = '低倍率 (<0.5)' THEN 1 ELSE 0 END) AS low_ratio_cnt,
    SUM(CASE WHEN ratio_tier = '常规病例 (0.5-2.0)' THEN 1 ELSE 0 END) AS normal_ratio_cnt,
    SUM(CASE WHEN ratio_tier = '高倍率 (2.0-3.0)' THEN 1 ELSE 0 END) AS high_ratio_cnt,
    SUM(CASE WHEN ratio_tier = '极高倍率 (>3.0)' THEN 1 ELSE 0 END) AS super_high_ratio_cnt
FROM CaseProfitLoss
GROUP BY dept_id
ORDER BY dept_net_profit ASC; -- 升序排列，亏损最严重的排在前面

```

SQL逻辑解读：此套SQL展示了数据转换的核心骨架。重点在于**CTE(Common Table Expressions)**的使用：通过第一步聚合出每个病例的实际总花费，第二步计算`cost_ratio`（即倍率）并进行硬编码的分层打标，最终在第三、四步实现各种维度的上卷（Roll-up）聚合。这正是BI工程师构建底层Data Mart（数据集市）的标准工序。

2.3 核心流程：病种盈亏分析六步法 (End-to-End Disease Profitability Analysis)

仅仅算出“亏了多少钱”属于描述性统计，高级运营分析的价值在于“找出亏损根因”并“预测未来趋势”。我们将一套严谨的数据科学项目流程引入医疗运营，提炼为**病种盈亏分析六步法**。

本节我们将提供一个**极其详尽的端到端 Python 实战案例**。我们将生成几千条模拟的单一病种（假定为：“DIP病种：I63 脑梗死，非手术，伴严重并发症”）住院数据，并完整执行这六步法。

六步法框架概览：

1. **数据清洗与特征工程 (Data Cleaning & Feature Engineering)**：处理缺失、异常、衍生出如“耗占比”、“术前天数”等关键特征。
2. **描述性统计分析 (Descriptive Statistics)**：大盘概貌，均值、极值、分布情况。
3. **相关性分析 (Correlation Analysis)**：探索哪些费用明细项目或患者属性与“盈亏金额”存在线性关系。
4. **分类比较分析 (Categorical Comparison - T检验/ANOVA)**：例如，不同主治医师的平均盈亏是否有显著差异？不同年龄段是否有差异？
5. **回归建模溯源 (Regression Modeling)**：构建多元线性回归（OLS），量化某个指标每增加1单位，对盈亏的具体金额影响，找出核心归因。
6. **机器学习预测 (Machine Learning for Prediction)**：利用随机森林等算法，基于入院前3天的特征，预测该患者出院时是否会产生医保亏损，实现**事前风控预警**。

Python 全景实战脚本

本脚本需依赖环境：pandas, numpy, matplotlib, seaborn, scipy, statsmodels, scikit-learn。

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.preprocessing import StandardScaler

# 设置中文字体（根据不同操作系统可能需要调整）
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

print("="*60)
print("开始执行：病种盈亏分析六步法深度演练")
print("分析目标病种：脑梗死（伴严重合并症）")
print("="*60)

# =====
# 步骤 0：构建高仿真 dummy dataset
```

```

# =====
np.random.seed(1024)
n_samples = 3000

# 假设该DIP病种的医保支付定额为 18000 元
benchmark_payment = 18000

data = {
    'patient_id': range(1, n_samples + 1),
    'age': np.random.normal(68, 12, n_samples).astype(int), # 年龄, 均值68
    'gender': np.random.choice(['M', 'F'], n_samples),
    'doctor_id': np.random.choice(['Doc_A', 'Doc_B', 'Doc_C', 'Doc_D'], n_samples, p=[0.3, 0.4,
    'length_of_stay': np.random.gamma(shape=5.0, scale=2.0, size=n_samples).astype(int) + 3, #
    # 各项费用模拟
    'drug_cost': np.random.lognormal(mean=8.5, sigma=0.5, size=n_samples), # 药品费
    'consumable_cost': np.random.lognormal(mean=7.5, sigma=0.8, size=n_samples), # 耗材费
    'service_cost': np.random.normal(loc=3000, scale=500, size=n_samples), # 医疗服务费(相对稳定)
    'exam_cost': np.random.normal(loc=2500, scale=800, size=n_samples), # 检查检验费
    'has_icu': np.random.choice([0, 1], n_samples, p=[0.8, 0.2]) # 是否进ICU
}
df = pd.DataFrame(data)

# 根据逻辑增加相关性: 住院天数越长, 费用越高; 进ICU耗材费大幅增加
df['service_cost'] = df['service_cost'] + df['length_of_stay'] * 200
df.loc[df['has_icu'] == 1, 'consumable_cost'] = df.loc[df['has_icu'] == 1, 'consumable_cost'] *
df.loc[df['has_icu'] == 1, 'length_of_stay'] = df.loc[df['has_icu'] == 1, 'length_of_stay'] + 5

# =====
# 步骤 1: 数据清洗与特征工程
# =====
print("\n[Step 1] 数据清洗与特征工程...")
# 处理不可能的负值和年龄极值
df['age'] = df['age'].clip(lower=20, upper=105)
for col in ['drug_cost', 'consumable_cost', 'service_cost', 'exam_cost']:
    df[col] = df[col].clip(lower=0)

# 特征衍生
df['total_cost'] = df['drug_cost'] + df['consumable_cost'] + df['service_cost'] + df['exam_cost']
df['drug_ratio'] = df['drug_cost'] / df['total_cost'] # 药占比
df['consumable_ratio'] = df['consumable_cost'] / df['total_cost'] # 耗占比

# 核心目标变量计算
df['profit'] = benchmark_payment - df['total_cost']
# 二分类目标变量: 是否亏损 (1: 亏损, 0: 盈余)
df['is_loss'] = (df['profit'] < 0).astype(int)

print(f"数据总览: 共 {len(df)} 条病例, 总体亏损比例为 {df['is_loss'].mean():.2%}")

```

```

# =====
# 步骤 2: 描述性统计
# =====
print("\n[Step 2] 描述性统计...")
desc_stats = df[['total_cost', 'length_of_stay', 'profit']].describe().round(2)
print("核心指标分布情况:\n", desc_stats)

# =====
# 步骤 3: 相关性分析
# =====
print("\n[Step 3] 变量相关性分析 (目标: 找出导致亏损/利润下降的量化指标)...")
# 选取数值型特征
num_features = ['age', 'length_of_stay', 'drug_cost', 'consumable_cost', 'service_cost', 'exam_
corr_matrix = df[num_features].corr()

# 提取与盈亏金额(profit)的相关系数
profit_corr = corr_matrix['profit'].sort_values()
print("各数值指标与'盈余金额'的皮尔逊相关系数 (负值表示该指标越大, 盈余越少): ")
print(profit_corr.drop('profit').to_frame().T.round(3))

# =====
# 步骤 4: 分类比较分析 (不同医生的控费差异)
# =====
print("\n[Step 4] 分类比较 (ANOVA): 不同主治医师的平均盈亏是否有显著差异?")
# 按医生分组看均值
doctor_profit = df.groupby('doctor_id')['profit'].mean().round(2)
print("各组医生平均盈亏:\n", doctor_profit)

# 单因素方差分析 (ANOVA)
doc_a = df[df['doctor_id'] == 'Doc_A']['profit']
doc_b = df[df['doctor_id'] == 'Doc_B']['profit']
doc_c = df[df['doctor_id'] == 'Doc_C']['profit']
doc_d = df[df['doctor_id'] == 'Doc_D']['profit']

f_stat, p_val = stats.f_oneway(doc_a, doc_b, doc_c, doc_d)
print(f"ANOVA F-statistic: {f_stat:.2f}, P-value: {p_val:.4e}")
if p_val < 0.05:
    print("-> 结论: 不同医生之间的控费能力/盈亏结果存在统计学意义上的【显著差异】。需要进一步下

# =====
# 步骤 5: 回归建模溯源 (OLS 归因分析)
# =====
print("\n[Step 5] 线性回归溯源: 量化各因素对利润的独立影响...")
# 定义自变量X (排除直接构成利润计算的绝对金额因子, 使用相对指标和客观指征)
X = df[['age', 'length_of_stay', 'drug_ratio', 'consumable_ratio', 'has_icu']]
# 添加常数项

```

```

X = sm.add_constant(X)
y = df['profit']

# 拟合 OLS 模型
ols_model = sm.OLS(y, X).fit()
print(ols_model.summary().tables[1]) # 仅打印系数表

print("\n-> 业务解读（以系数估计值为准）：")
print(" 1. 住院天数(length_of_stay)系数显著为负：即在其他条件不变下，患者每多住1天，科室平均利
print(" 2. 是否进ICU(has_icu)系数为负：患者一旦进入ICU，会导致该DIP病例平均亏损增加约 {:.0f} 元

# =====
# 步骤 6：机器学习风控预警（随机森林分类）
# =====
print("\n[Step 6] 机器学习预测模型：入院早期预知亏损风险（事前风控）")
# 假设场景：我们要在患者入院第3天就预测其最终是否会亏损（以便及时干预）
# 特征我们仅使用患者基础信息 + 早期能知道的信息
features_for_ml = ['age', 'gender', 'doctor_id', 'has_icu']
X_ml = pd.get_dummies(df[features_for_ml], drop_first=True) # 对类别变量进行独热编码（One-Hot）
y_ml = df['is_loss']

# 划分训练集和测试集（80% / 20%）
X_train, X_test, y_train, y_test = train_test_split(X_ml, y_ml, test_size=0.2, random_state=42)

# 特征标准化（对树模型不是必须，但为良好习惯）
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 构建随机森林模型并训练
rf = RandomForestClassifier(n_estimators=100, max_depth=5, class_weight='balanced', random_state=42)
rf.fit(X_train_scaled, y_train)

# 预测
y_pred = rf.predict(X_test_scaled)
y_prob = rf.predict_proba(X_test_scaled)[:, 1] # 预测为亏损的概率

# 评估模型
print("\n预测模型评估报告（Classification Report）:")
print(classification_report(y_test, y_pred))

auc_score = roc_auc_score(y_test, y_prob)
print(f"-> 模型 AUC 值：{auc_score:.3f}（大于0.7表示有较好的区分度，可部署到HIS系统中进行实时弹

# 输出特征重要性
importance = pd.DataFrame({'Feature': X_ml.columns, 'Importance': rf.feature_importances_})
importance = importance.sort_values(by='Importance', ascending=False)

```

```
print("\n-> 预警核心特征重要度排序：")
print(importance)

print("\n==== 演练完成 ====")
```

深入解读：六步法的业务意义

上述六个步骤构成了一个由浅入深、由表及里的闭环：

1. **第一二步（清洗与描述）** 解决了“**是什么**”的问题：让我们知道科室现在到底亏不亏，亏的大盘面貌如何。
2. **第三四步（相关与分类比较）** 解决了“**哪里出了问题**”的问题：是特定高龄人群亏？还是某个习惯大处方的医生导致的亏？通过假设检验（ANOVA），我们将“感觉上XX医生花费多”变成了具备统计学铁证的科学结论。
3. **第五步（多元回归）** 解决了“**影响有多大**”的问题：剥离混杂因素。单纯看住院天数可能不准（因为重症住得长），但多元回归（OLS）能在“固定患者年龄、是否进ICU等条件相同”的数学前提下，孤立出“纯粹由于多住一天导致利润下降的确切金额”。这就是精细化管理的抓手。
4. **第六步（机器学习）** 完成了从“**事后算账**”向“**事前风控**”的革命性跨越。传统的运营分析总是看上个月的报表，而通过训练随机森林（Random Forest）或XGBoost模型，我们可以在患者刚入院或手术刚结束时，将其年龄、基础疾病、早期化验指标等输入模型。一旦模型输出“亏损概率 > 80%”，HIS系统即可向主治医生弹窗预警，建议其在保证医疗安全的前提下，严格控制可替代的高值耗材使用，甚至提示申请医保特例单议。这就是**数据赋能临床**的终极形态。